

# Intermediate SQL Tasks & Solutions

17 hands-on tasks built directly on the **MyDatabase** schema — joins, window functions, CTEs, subqueries, and conditional aggregation — each with a full T-SQL solution and explanation.

**Tables used:** customers (id, first\_name, country, score) · orders (order\_id, customer\_id, order\_date, sales) · employees (id, first\_name, last\_name, department, salary)

17 Tasks

T-SQL / SQL Server

Window Fns · CTEs · Subqueries

# About This Guide

---

This guide contains 17 intermediate-level SQL exercises built on top of the **MyDatabase** database created by the setup script — three tables, 500 rows each. Every task assumes the schema below and is written in T-SQL (SQL Server), matching the dialect used in the original creation script.

Each task follows the same structure: a task description, a complete working solution in a styled code panel, and a line-by-line explanation of the key techniques involved. Several tasks also include an "Interview Insight" callout highlighting how the concept tends to be tested in real interviews.

| Table            | Columns                                       | Rows |
|------------------|-----------------------------------------------|------|
| <b>customers</b> | id, first_name, country, score                | 500  |
| <b>orders</b>    | order_id, customer_id, order_date, sales      | 500  |
| <b>employees</b> | id, first_name, last_name, department, salary | 500  |

**Topics covered:** multi-table joins, GROUP BY & HAVING, window functions (RANK, ROW\_NUMBER, LAG, SUM OVER), common table expressions (CTEs), scalar and correlated subqueries, EXISTS, CASE expressions, string and date functions, and conditional aggregation (manual pivoting).

# Task Index

|    |                                                                  |                                         |
|----|------------------------------------------------------------------|-----------------------------------------|
| 01 | Customer Spending Summary                                        | JOIN • GROUP BY • HAVING                |
| 02 | Country-Level Score Statistics                                   | GROUP BY • Aggregate Functions          |
| 03 | Rank Customers Within Their Country                              | Window Functions • RANK()               |
| 04 | Above-Average Score Customers                                    | Subqueries                              |
| 05 | Monthly Sales Trend                                              | CTE • Date Functions                    |
| 06 | Employees Above Department Average                               | Window Functions • AVG() OVER           |
| 07 | Employee Full Names and Initials                                 | String Functions                        |
| 08 | Orders by Quarter with Recency                                   | Date Functions • DATEDIFF               |
| 09 | Customer Score Segmentation                                      | CASE Expressions                        |
| 10 | High-Value Customer Orders                                       | Multi-Table JOIN • Filtering            |
| 11 | Salary Gap From the Previous Colleague                           | Window Functions • LAG()                |
| 12 | Each Customer's Most Recent Order                                | Correlated Subquery                     |
| 13 | Customers With a Large Single Order                              | EXISTS                                  |
| 14 | Top 3 Highest-Paid Employees per Department                      | Window Functions • ROW_NUMBER() • CTE   |
| 15 | Each Customer's Share of Total Sales                             | Window Functions • SUM() OVER()         |
| 16 | Department Headcount and Salary Budget                           | GROUP BY • Multiple Aggregates • HAVING |
| 17 | Customer Count by Country (Pivoted with Conditional Aggregation) | Conditional Aggregation • CASE + SUM    |

## 01

## Customer Spending Summary

JOIN • GROUP BY • HAVING

## TASK

For each customer, calculate their total number of orders and total sales amount. Only include customers whose total sales exceed 5000. Sort the result by total sales in descending order.

## SOLUTION

```
T-SQL

SELECT
  c.id                AS customer_id,
  c.first_name,
  c.country,
  COUNT(o.order_id)   AS total_orders,
  SUM(o.sales)        AS total_sales
FROM customers AS c
JOIN orders AS o
  ON o.customer_id = c.id
GROUP BY
  c.id,
  c.first_name,
  c.country
HAVING SUM(o.sales) > 5000
ORDER BY total_sales DESC;
```

## EXPLANATION

- The **JOIN** links every order to the customer who placed it, so aggregate functions can be computed per customer.
- **GROUP BY** must list every non-aggregated column in the SELECT list — here that's `id`, `first_name`, and `country`.
- **HAVING** filters on the aggregated result ( `SUM(o.sales)` ); **WHERE** cannot be used here because the filter depends on a group total.

## INTERVIEW INSIGHT

Interviewers often test whether you know the difference between **WHERE** (filters rows before grouping) and **HAVING** (filters groups after aggregation). Mixing these up is one of the most common SQL interview mistakes.

## TASK

For each country, report the number of customers, the average score (rounded to 1 decimal place), and the minimum and maximum score. Only include countries with at least 50 customers.

## SOLUTION

```
T-SQL

SELECT
    country,
    COUNT(*)                AS customer_count,
    ROUND(AVG(CAST(score AS FLOAT)), 1) AS avg_score,
    MIN(score)              AS min_score,
    MAX(score)              AS max_score
FROM customers
GROUP BY country
HAVING COUNT(*) >= 50
ORDER BY avg_score DESC;
```

## EXPLANATION

- `score` is an `INT` column, so it's cast to `FLOAT` before averaging — otherwise SQL Server performs integer division and truncates the decimal portion.
- Multiple aggregates ( `COUNT` , `AVG` , `MIN` , `MAX` ) can be computed in a single pass over each group.
- `ROUND(value, 1)` keeps the report readable without losing meaningful precision.

## TASK

Rank customers by score within each country, with rank 1 being the highest score. Return the customer id, name, country, score, and their rank.

## SOLUTION

```
T-SQL

SELECT
  id,
  first_name,
  country,
  score,
  RANK() OVER (
    PARTITION BY country
    ORDER BY score DESC
  ) AS score_rank
FROM customers
ORDER BY country, score_rank;
```

## EXPLANATION

- `PARTITION BY country` resets the ranking for every country, so each country has its own rank 1.
- `RANK()` assigns the same rank to tied scores and then skips the next rank number (e.g. 1, 2, 2, 4). Use `DENSE_RANK()` instead if you want no gaps, or `ROW_NUMBER()` if ties should be broken arbitrarily.
- Window functions run after the `WHERE` clause but do not collapse rows the way `GROUP BY` does — every original row is preserved.

## INTERVIEW INSIGHT

Know the three ranking functions cold: `ROW_NUMBER()` (always unique, no ties), `RANK()` (ties share a rank, gaps follow), and `DENSE_RANK()` (ties share a rank, no gaps). This is a very common whiteboard question.

## TASK

Find all customers whose score is higher than the overall average score across all customers.

## SOLUTION

```
T-SQL

SELECT
  id,
  first_name,
  country,
  score
FROM customers
WHERE score > (
  SELECT AVG(CAST(score AS FLOAT))
  FROM customers
)
ORDER BY score DESC;
```

## EXPLANATION

- The subquery in the `WHERE` clause is a scalar subquery — it returns exactly one value (the overall average), which is then compared against every row's `score`.
- Because the subquery does not reference the outer query, it only needs to be evaluated once, not once per row (a non-correlated subquery).
- This same result could be written with a window function ( `AVG(score) OVER ( )` ) and an outer filter, which is often more efficient for large tables.

## TASK

Using a CTE, calculate the total sales and number of orders for each calendar month. Return the year, month, total sales, and order count, sorted chronologically.

## SOLUTION

```
T-SQL

WITH MonthlySales AS (
    SELECT
        YEAR(order_date) AS order_year,
        MONTH(order_date) AS order_month,
        sales
    FROM orders
)
SELECT
    order_year,
    order_month,
    COUNT(*) AS total_orders,
    SUM(sales) AS total_sales
FROM MonthlySales
GROUP BY order_year, order_month
ORDER BY order_year, order_month;
```

## EXPLANATION

- The CTE ( `MonthlySales` ) pre-extracts the year and month from each order's date, keeping the final `GROUP BY` clean and readable.
- `YEAR()` and `MONTH()` are SQL Server date functions that pull out date parts; `DATEPART(month, order_date)` is an equivalent alternative.
- CTEs are especially useful here because they let you build a query in logical stages instead of nesting subqueries.

## TASK

List every employee along with their department's average salary, and flag whether that employee earns above their department average.

## SOLUTION

```
T-SQL

SELECT
    id,
    first_name,
    last_name,
    department,
    salary,
    ROUND(AVG(CAST(salary AS FLOAT)) OVER (
        PARTITION BY department
    ), 0) AS dept_avg_salary,
    CASE
        WHEN salary > AVG(CAST(salary AS FLOAT)) OVER (PARTITION BY department)
        THEN 'Above Average'
        ELSE 'At or Below Average'
    END AS salary_flag
FROM employees
ORDER BY department, salary DESC;
```

## EXPLANATION

- `AVG(salary) OVER (PARTITION BY department)` computes each department's average without collapsing rows, so individual employee detail is preserved alongside the group statistic.
- This avoids a self-join or a correlated subquery, both of which are valid but less efficient alternatives for the same result.
- The `CASE` expression reuses the same window calculation to classify each row inline.

## INTERVIEW INSIGHT

A very common follow-up interview question: 'How would you write this without a window function?' Be ready to show the correlated-subquery version too:

```
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e2.department = e1.department)
```

## TASK

Generate a report showing each employee's full name, their initials (e.g. 'J.S.'), and the length of their full name.

## SOLUTION

```
SELECT
  id,
  first_name + ' ' + last_name AS full_name,
  LEFT(first_name, 1) + '.' + LEFT(last_name, 1) + '.' AS initials,
  LEN(first_name + ' ' + last_name) AS name_length
FROM employees
ORDER BY last_name, first_name;
```

## EXPLANATION

- In T-SQL, `+` concatenates strings (unlike MySQL, which requires `CONCAT()`). `CONCAT(first_name, ' ', last_name)` also works and additionally handles NULLs gracefully.
- `LEFT(string, n)` extracts the first `n` characters, used here to build initials from each name.
- `LEN()` returns character count, but note it trims trailing spaces — use `DATALength()` if trailing whitespace matters.

## TASK

For every order, show which quarter of the year it fell in, and how many days ago it was placed relative to 2022-01-01.

## SOLUTION

T-SQL

```
SELECT
    order_id,
    customer_id,
    order_date,
    DATEPART(QUARTER, order_date) AS order_quarter,
    DATEDIFF(DAY, order_date, '2022-01-01') AS days_before_2022
FROM orders
ORDER BY order_date;
```

## EXPLANATION

- `DATEPART(QUARTER, date)` returns 1-4 depending on which quarter the date falls in.
- `DATEDIFF(DAY, start_date, end_date)` returns the number of days between two dates; the sign depends on argument order, so double-check which direction you need.
- Storing computed columns like this as a view is a common pattern when the same date logic is reused across several reports.

## TASK

Classify each customer into a score band: 'Low' (0-333), 'Medium' (334-666), or 'High' (667-1000). Then count how many customers fall into each band.

## SOLUTION

```
T-SQL

SELECT
  CASE
    WHEN score <= 333 THEN 'Low'
    WHEN score BETWEEN 334 AND 666 THEN 'Medium'
    ELSE 'High'
  END AS score_band,
  COUNT(*) AS customer_count
FROM customers
GROUP BY
  CASE
    WHEN score <= 333 THEN 'Low'
    WHEN score BETWEEN 334 AND 666 THEN 'Medium'
    ELSE 'High'
  END
ORDER BY
  MIN(score);
```

## EXPLANATION

- A `CASE` expression can be used directly inside `GROUP BY` — SQL Server groups rows by the resulting label, not the underlying column.
- The same `CASE` logic has to be repeated in both the `SELECT` list and the `GROUP BY` clause since column aliases can't be reused within the same query level in T-SQL.
- Ordering by `MIN(score)` keeps the bands in a logical Low → Medium → High sequence instead of alphabetical order.

## INTERVIEW INSIGHT

A cleaner alternative is to wrap the `CASE` logic in a CTE first, then `GROUP BY` the resulting column name once — avoids repeating the expression.

## TASK

Retrieve the full name, country, score, order date, and sales amount for every order placed by customers whose score is above 700, where the order's sales amount also exceeds 500.

## SOLUTION

```
T-SQL

SELECT
    c.first_name,
    c.country,
    c.score,
    o.order_date,
    o.sales
FROM customers AS c
JOIN orders AS o
    ON o.customer_id = c.id
WHERE c.score > 700
    AND o.sales > 500
ORDER BY o.sales DESC;
```

## EXPLANATION

- Filtering on columns from both tables ( `c.score` and `o.sales` ) is only possible after the join has combined the rows.
- Placing selective filters in the `WHERE` clause lets the query optimizer reduce the row count as early as possible in the execution plan.
- Table aliases ( `c` , `o` ) keep multi-table queries readable and are required once a column name could be ambiguous.

## TASK

Within each department, order employees by salary ascending, then show how much more each employee earns than the colleague immediately below them in that ordering.

## SOLUTION

```
T-SQL

SELECT
    department,
    first_name,
    last_name,
    salary,
    salary - LAG(salary) OVER (
        PARTITION BY department
        ORDER BY salary
    ) AS gap_from_previous
FROM employees
ORDER BY department, salary;
```

## EXPLANATION

- `LAG(salary)` looks at the salary value from the previous row within the same partition (department), based on the specified `ORDER BY`.
- The first employee in each department has no 'previous' row, so `gap_from_previous` returns `NULL` for them — this is expected behavior, not a bug.
- `LEAD()` is the mirror image of `LAG()`, looking forward instead of backward; both are common for period-over-period comparisons.

## INTERVIEW INSIGHT

LAG/LEAD questions often appear framed as 'month-over-month growth' or 'previous transaction comparison' — the pattern is identical to this task.

## TASK

For every customer who has placed at least one order, return their most recent order date.

## SOLUTION

```
SELECT
  c.id,
  c.first_name,
  (
    SELECT MAX(o.order_date)
    FROM orders AS o
    WHERE o.customer_id = c.id
  ) AS most_recent_order
FROM customers AS c
WHERE EXISTS (
  SELECT 1
  FROM orders AS o2
  WHERE o2.customer_id = c.id
);
```

## EXPLANATION

- This is a correlated subquery: the inner query references `c.id` from the outer query, so it re-executes conceptually once per outer row.
- The outer `EXISTS` filter excludes customers with zero orders entirely, which avoids returning rows with a `NULL` order date.
- The equivalent, often more efficient, way to write this is a plain `JOIN` combined with `GROUP BY c.id, c.first_name` and `MAX(o.order_date)`.

## TASK

Return the customers who have placed at least one order with sales greater than 900, without duplicating a customer row for each such order.

## SOLUTION

```
T-SQL

SELECT
    c.id,
    c.first_name,
    c.country
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id
        AND o.sales > 900
)
ORDER BY c.id;
```

## EXPLANATION

- **EXISTS** stops scanning as soon as it finds one matching row, making it efficient for 'does at least one match exist' questions.
- Unlike a **JOIN** + **DISTINCT** approach, **EXISTS** naturally returns each customer only once, with no risk of row duplication.
- **SELECT 1** inside the subquery is a convention — the actual selected value is irrelevant since **EXISTS** only checks for row presence.

## TASK

Find the top 3 highest-paid employees within each department.

## SOLUTION

```
WITH RankedEmployees AS (  
  SELECT  
    id,  
    first_name,  
    last_name,  
    department,  
    salary,  
    ROW_NUMBER() OVER (  
      PARTITION BY department  
      ORDER BY salary DESC  
    ) AS salary_rank  
  FROM employees  
)  
SELECT  
  department,  
  first_name,  
  last_name,  
  salary,  
  salary_rank  
FROM RankedEmployees  
WHERE salary_rank <= 3  
ORDER BY department, salary_rank;
```

## EXPLANATION

- Window functions cannot be referenced directly in a `WHERE` clause in the same query level, which is exactly why a CTE (or subquery) is required here.
- `ROW_NUMBER()` guarantees exactly one row per rank per department, even if two employees share the same salary — useful when you need a strict top-N count.
- This 'top N per group' pattern is one of the most frequently asked SQL interview problems.

## INTERVIEW INSIGHT

If ties should all be included (e.g. two employees tied for 3rd place both appear), swap `ROW_NUMBER()` for `RANK()` instead.

## TASK

For every order, show what percentage of the grand total sales that single order represents, rounded to 2 decimal places.

## SOLUTION

```
T-SQL

SELECT
  order_id,
  customer_id,
  sales,
  ROUND(
    100.0 * sales / SUM(sales) OVER (),
    2
  ) AS pct_of_total_sales
FROM orders
ORDER BY pct_of_total_sales DESC;
```

## EXPLANATION

- `SUM(sales) OVER ()` with an empty `OVER()` clause computes the grand total across the entire result set, while still returning one row per order.
- Multiplying by `100.0` (not `100`) forces floating-point division, avoiding the integer-division truncation that would silently zero out the percentage.
- This pattern replaces what would otherwise require a self-join against a pre-aggregated total, or two separate queries combined in application code.

## TASK

For each department, report the number of employees, the total salary budget, and the average salary (rounded to 0 decimals). Only include departments with an average salary above 5000.

## SOLUTION

```
T-SQL

SELECT
    department,
    COUNT(*)                AS headcount,
    SUM(salary)              AS total_salary_budget,
    ROUND(AVG(CAST(salary AS FLOAT)), 0) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(CAST(salary AS FLOAT)) > 5000
ORDER BY total_salary_budget DESC;
```

## EXPLANATION

- Casting to `FLOAT` before `AVG()` prevents integer truncation, the same issue as in earlier tasks — worth building as a habit for any INT column average.
- The `HAVING` condition repeats the aggregate expression from the `SELECT` list; column aliases from `SELECT` cannot be reused inside `HAVING` in T-SQL.
- This is a typical 'budget report' style query — a common real-world use case for `GROUP BY` with `HAVING`.

## TASK

Produce a single summary row showing the count of customers in Egypt, USA, and Germany as three separate columns (a manual pivot).

## SOLUTION

```
T-SQL

SELECT
    SUM(CASE WHEN country = 'Egypt' THEN 1 ELSE 0 END) AS egypt_customers,
    SUM(CASE WHEN country = 'USA' THEN 1 ELSE 0 END) AS usa_customers,
    SUM(CASE WHEN country = 'Germany' THEN 1 ELSE 0 END) AS germany_customers
FROM customers;
```

## EXPLANATION

- This 'conditional aggregation' technique converts row values into separate output columns without using SQL Server's dedicated `PIVOT` operator.
- Each `CASE` expression evaluates to 1 or 0 per row, and `SUM()` then totals up the matching rows for that specific country.
- It's a very portable pattern — the same logic works unchanged in MySQL, Postgres, or any SQL dialect, whereas SQL Server's native `PIVOT` syntax does not.

## INTERVIEW INSIGHT

This exact `CASE + SUM` technique is worth memorizing — it comes up constantly whenever an interviewer asks you to 'turn rows into columns' without specifying a particular `PIVOT` syntax.

## Key Takeaways

- Window functions (`OVER()`, `PARTITION BY`) let you compute aggregates without collapsing rows — the single most important skill jump from beginner to intermediate SQL.
- Always be explicit about `WHERE` vs `HAVING`: filter individual rows before grouping, filter aggregated groups after.
- Cast `INT` columns to `FLOAT` before averaging to avoid silent integer-division truncation — a very common bug in real reporting queries.
- CTEs make multi-step logic (like ranking-then-filtering) both required and far more readable than deeply nested subqueries.
- `EXISTS` is generally more efficient than `JOIN + DISTINCT` for "at least one match" questions, since it stops scanning early.